INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
DEPARTMENT OF MECHATRONICS ENGINEERING

MECHATRONIC SYSTEM INTEGRATION (MCTA 3203)
SEMESTER 1, 2026/2027

EXPERIMENT 2: SERIAL COMMUNICATION

SECTION: 1 | GROUP: 11
SUBMISSION DATE: 18/10/2026

NO.	MATRIC NO.	NAME
1	2414273	DANISH IMAN BIN AJO
2	2414585	ALIFF EMIR BIN AHMAD EZAD
3	2416879	AHMAD AMIRULAQIL BIN ZAINORKHAIR
4	2418895	MUHAMMAD IRFAN BIN ROSHDAN

ABSTRACT

The purpose of these experiments is to both read potentiometer values and control a servo motor via serial communication. This was achieved by using a Serial USB as a means of communication between the Arduino UNO microcontroller and the computer. Python was used to upload codes and communicate with the Arduino UNO to control the LED and servo motor. Additional tasks such as connecting an LED to the potentiometer to create an off and on switch based on a threshold value and connecting the servo motor to the potentiometer for more intuitive control. The experiment begins with connecting the potentiometer to the Arduino UNO and other components such as the LED and servo motor will be connected in later tasks. A code to read the potentiometer values is uploaded and the potentiometer values can be read through the Serial Monitor where the value ranges from 0 to 1023. Then, an LED is connected to the Arduino UNO where it can be turned on and off by reading the potentiometer values and setting a threshold for the switch control. The same is done with the servo motor where it is connected to the Arduino UNO and manipulated using the potentiometer values.

TABLE OF CONTENTS

ABSTRACT 2
TABLE OF CONTENTS 2
PART 1 2
- **1.0 INTRODUCTION & THEORY** 2
- **1.1 MATERIALS AND EQUIPMENT** 2
- **1.2 EXPERIMENTAL SETUP** 2
- **1.3 METHODOLOGY & PROGRAMMING LOGIC** 4
- **1.4 DATA COLLECTION & ANALYSIS** 4
- **1.5 RESULTS & DISCUSSION** 4

PART 2 4
- **2.0 INTRODUCTION & THEORY** 4
- **2.1 MATERIALS AND EQUIPMENT** 4
- **2.2 EXPERIMENTAL SETUP** 4
- **2.3 METHODOLOGY & PROGRAMMING LOGIC** 5
- **2.4 DATA COLLECTION & ANALYSIS** 5
- **2.5 RESULTS & DISCUSSION** 5

RECOMMENDATIONS 5
REFERENCES 5
ACKNOWLEDGEMENT 5
STUDENT DECLARATION 5

PART 1

1.0 INTRODUCTION & THEORY

This part of the experiment involves two core components which are the potentiometer and LED and will be conducted using two different IDEs being Arduino IDE and PyCharm for the integration of Python programming. This experiment aims to achieve proper reading of potentiometer values via serial communication which is established by using the USB connected from the Arduino UNO to the computer as a means of data transfer. It should be noted that the *pyserial* library is to be installed first to establish connection between PyCharm and the Arduino UNO.

The potentiometer is connected to the Arduino UNO by wiring the center pin of the potentiometer to an unoccupied analog pin and connecting both left and right pins to the GND and 5V respectively. After that, the potentiometer can be adjusted by turning its knob to change its resistance, thus altering the amount of current transferred along the analog pin. A code is uploaded to the Arduino UNO to read and send the current signal to the serial monitor whereby the signal ranges from 0 to 1023.

After successfully completing this experiment, the first task can be carried out by connecting an LED with a 220 ohm connected in series to the Arduino UNO. The potentiometer signal value is read and an if-else statement is used to read whether the reading has passed its half-mark threshold or not to turn the LED on and off accordingly. Then, the potentiometer readings can be read either in the PyCharm terminal or the Arduino IDE serial plotter to view a graphical representation of the readings but the Arduino IDE may not be used simultaneously with PyCharm to avoid any interruption between the Python script and the Arduino UNO.

1.1 MATERIALS AND EQUIPMENT

- Arduino UNO board
- Potentiometer
- Jumper wires
- Breadboard
- LED
- A computer
- Arduino IDE installed
- PyCharm with pyserial library installed

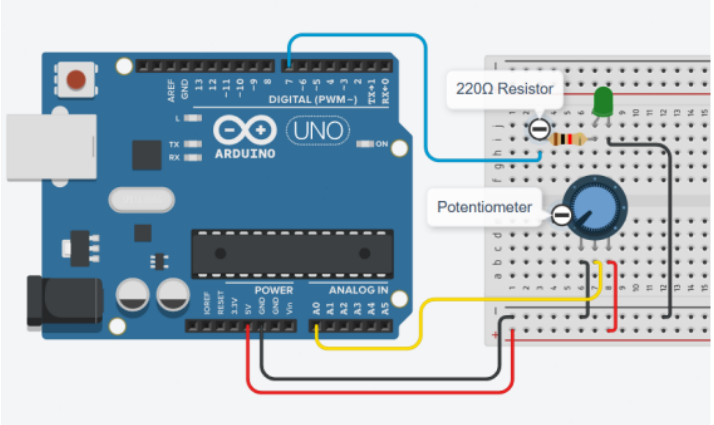
1.2 EXPERIMENTAL SETUP

1. Connect the Arduino to your computer via a USB cable.

2. Extend the circuit by adding an LED to digital pin 7 in series with a 220 Ω resistor connected to GND.

3. We will now establish bidirectional communication. The Arduino will send the potentiometer values to Python, and Python will send a command back to turn the LED ON if the value exceeds half its maximum range (ADC > 512).

Circuit diagram



1.3 METHODOLOGY & PROGRAMMING LOGIC

Experiment Steps:

1. Connect the Arduino UNO to the computer using a USB cable
2. Upload the Arduino sketch to the Arduino UNO using the Arduino IDE
3. Run the Python script using the PyCharm IDE with pyserial library installed

   The pyserial library is installed by running this following command line in the terminal

   ```
   pip install pyserial
   ```

   If any error arises stating the command is unrecognizable then an interpreter is needed before running the command once more. The interpreter may be installed by opening *Settings* then under *Python* choose *Interpreter* and select *Add Interpreter*, *Add Local Interpreter* and choosing *Virtualenv* type.

4. The potentiometer knob is turned to tune the signal sent to the Python terminal whereby it can be read as a value ranging from 0 to 1023.
5. These readings may be used for various projects involving data logging or control applications depending on the project's requirements

   Before continuing, it should be noted however that the Arduino IDE serial plotter must not be used simultaneously while working with Python. The Arduino IDE serial plotter requires uninterrupted use of the serial port so it may interfere with Python's usage for the terminal. It is recommended that the Arduino IDE is closed while the Python script is running to ensure that no interruption between Python and the Arduino UNO occurs.

6. Open the serial plotter in the Arduino IDE by going to *Tools* and selecting *Serial Plotter*.

7. Ensure that the correct COM port is selected in the *Serial Plotter*.
8. Set the Baud Rate of the *Serial Plotter* to the one set in the code.
9. The *Serial Plotter* automatically displays a graphical representation of the potentiometer readings in real-time.
10. The *Serial Plotter* may also be customized by adjusting settings such as the graph range, labels, and colours.

After these steps are successfully completed, the first task may be carried out.

1. An LED is connected to the Arduino UNO following the circuit setup.
2. Upload the provided Python code to the Arduino UNO
3. The potentiometer knob is turned and the LED will turn on and off accordingly to a threshold value set in the Python code.

Source Code Blocks:

Arduino Code Block:

Experiment:

```
int potPin = A0;
int ledPin = 8;
int potValue = 0;

void setup() {
     Serial.begin(9600);
}
void loop() {
     int potValue = analogRead(A0);
     Serial.println(potValue);
     delay(1000); // add a delay to avoid sending data too
fast
}
```

Task 1:

```
int potPin = A0;
int ledPin = 8;
int potValue = 0;
String command = "";

void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
}

void loop() {

  potValue = analogRead(potPin);
  Serial.println(potValue);


  if (Serial.available() > 0) {
    command = Serial.readStringUntil('\n');
    command.trim();

    if (command == "ON") {
      digitalWrite(ledPin, HIGH);
    } else if (command == "OFF") {
      digitalWrite(ledPin, LOW);
    }
  }
}
```

Python Code Block:

Experiment:

```
import serial
# -- Replace 'COMx' with your Arduino's serial port
ser = serial.Serial('COMx', 9600)
try:
     while True:
          pot_value = ser.readline().decode().strip()
          print("Potentiometer Value:", pot_value)
except KeyboardInterrupt:
     ser.close()
     print("Serial connection closed.")
```

Task 1:

```
import serial
import time

ser = serial.Serial('COM7', 9600, timeout=0.1)
time.sleep(2)

try:
    while True:
        line = ser.readline().decode().strip()
        if line.isdigit():
            pot_value = int(line)
            print("Potentiometer Value:", pot_value)


            if pot_value > 512:
                ser.write(b"ON\n")
            else:
                ser.write(b"OFF\n")

except KeyboardInterrupt:
    print("\nProgram interrupted by user.")

finally:
    ser.close()
    print("Serial connection closed.")
```

1.4 DATA COLLECTION & ANALYSIS

The data confirmed that:

1. The potentiometer and the LED are working properly.
2. The Python logic controls the LED condition instead of Arduino IDE.

1.5 RESULTS & DISCUSSION

Both the experiment and task given were able to be carried out without any issues with the hardware. The communication between the Arduino UNO and computer was properly established and we were able to use the Arduino UNO with the Arduino IDE and PyCharm IDE. Besides that, we managed to control the LED with the potentiometer acting as an on and off switch based on a threshold value set in the code. Finally, we only encountered issues in the beginning of the experiment with the PyCharm IDE because the Python Interpreter was not set up before installing the *pyserial* library, causing us not being able to upload the Python script to the Arduino UNO.

1.6 CONCLUSION

Overall, this experiment was a success as our objective to transmit potentiometer readings from Arduino to Python using a USB serial connection was achieved. The task given which was to display potentiometer readings in the Python terminal and graphically represent the potentiometer readings in real-time was also achieved. Additionally, the task required an LED to be connected to the Arduino UNO and controlled with the potentiometer acting as an on and off switch based on the threshold given was also completed.

PART 2

2.0 INTRODUCTION & THEORY

The second part is a continuation of the first part with an addition of another core component being the servo motor. This part requires achieving control of servo motor angles using inputs from Python transmitted to the Arduino and visualizing sensor data graphically on the computer.

Technically, the Arduino UNO is able to receive instructions from a Python script via serial communication using a serial USB as a means of data transfer, so the Python terminal can be used to give out instructions such as an input angle and transmit it to the Arduino UNO for controlling the servo motor.

Additionally, the potentiometer was used to control the servo motor according to the potentiometer knob's rotation and an LED was added as additional feedback. Graphical visualization of the potentiometer readings is done using the *matplotlib* library in PyCharm.

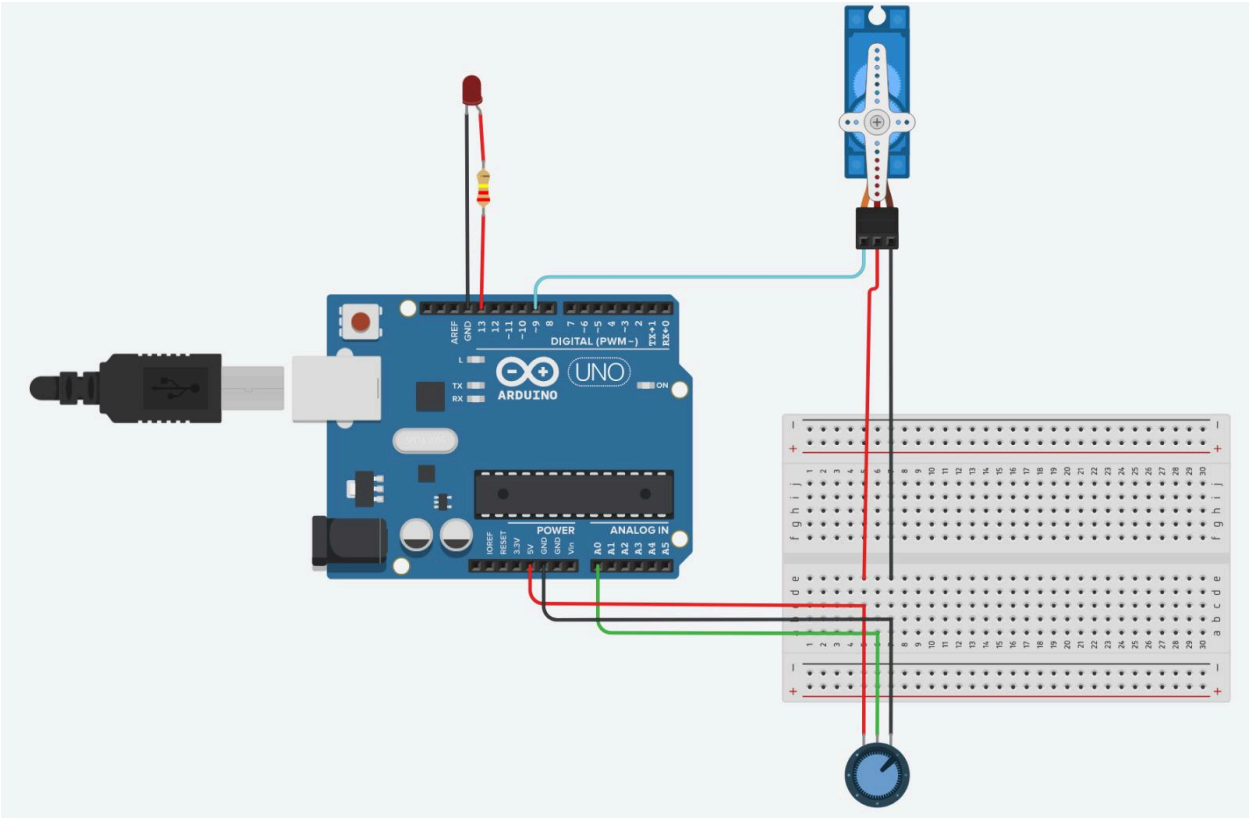
2.1 MATERIALS AND EQUIPMENT

- Arduino Board

- Servo Motor
- Jumper Wires
- Potentiometer
- LED
- Breadboard
- A computer
- Arduino IDE installed
- PyCharm with pyserial library installed

2.2 EXPERIMENTAL SETUP

1. Connect the LED to the Arduino Uno as follows:
    - Connect the anode (positive leg) of the red LED to digital pin D13 on the Arduino
    - Connect the cathode (negative leg) of the LED to the GND (ground) pin adjacent to D13 on the Arduino.
    - Use a 220-ohm resistor in series between the LED anode and digital pin D13 to limit the current.
2. Set up the breadboard power distribution from the Arduino:
    - Connect the 5V output pin of the Arduino to a terminal column (column 5) on the breadboard.
    - Connect one of the GND (ground) pins from the POWER header on the Arduino to a separate terminal column (column 7) on the breadboard to create shared power and ground nodes.
3. Connect the micro servo motor to the Arduino and breadboard:
    - Connect the signal pin (orange wire) of the servo motor directly to PWM digital pin D9 on the Arduino.
    - Connect the power pin (red wire) of the servo motor to the shared 5V column on the breadboard.
    - Connect the ground pin (brown/black wire) of the servo motor to the shared GND column on the breadboard.
4. Connect the potentiometer to the Arduino and breadboard:
    - Connect the left outer terminal of the potentiometer to the shared 5V column on the breadboard.
    - Connect the right outer terminal of the potentiometer to the shared GND column on the breadboard.
    - Connect the middle wiper pin of the potentiometer to analog input pin A0 on the Arduino via the breadboard.



2.3 METHODOLOGY & PROGRAMMING LOGIC

1. Enhance the Arduino and Python code to incorporate a potentiometer for real-time adjustments of the servo motor's angle. In the updated Arduino code, the program can be executed by pressing a designated key on the computer's keyboard. Following the modification, restart the Python script to receive and display servo position data from the Arduino over the serial connection. While experimenting with the potentiometer, observe the corresponding changes in the servo motor's position.
2. Enhance the experiment by adding an LED that lights up based on the potentiometer value or servo position. This adds a further layer of control and feedback.
3. Integrate graphical visualization using matplotlib to plot potentiometer values in real-time.

Here is the full code:

```
import serial
import matplotlib.pyplot as plt
import time

# Connect to your active port
ser = serial.Serial('COM4', 9600, timeout=0.01)
time.sleep(2)

# Set up Matplotlib for HIGH-SPEED interactive plotting
plt.ion()
```

```
fig, ax = plt.subplots()
x_vals, y_vals = [0] * 50, [0] * 50
line, = ax.plot(x_vals, y_vals, color='blue')
ax.set_ylim(0, 1023)
ax.set_title("Real-Time Potentiometer Data")
ax.set_ylabel("Analog Value (0-1023)")

smoothed_pot_value = None
alpha = 0.4

def remap(x, in_min, in_max, out_min, out_max):
    return (x - in_min) * (out_max - out_min) / (in_max -
in_min) + out_min

def on_press(event):
    if event.key == 'h':
        ser.write(b'h\n')
        print(">>> HALT COMMAND SENT")
    elif event.key == 'r':
        ser.write(b'r\n')
        print(">>> RESUME COMMAND SENT")

fig.canvas.mpl_connect('key_press_event', on_press)

try:
    print("Starting real-time plot. Press 'h' to halt, 'r' to
resume.")
    counter = 0

    while True:
        latest_line = None
        # Read all available lines in the buffer and keep only
the newest one
        while ser.in_waiting > 0:
            latest_line =
ser.readline().decode('utf-8').strip()

        if latest_line and latest_line.isdigit():
            raw_pot_value = int(latest_line)

            # Fixed Smoothing Math
            if smoothed_pot_value is None:
                smoothed_pot_value = raw_pot_value
            else:
                smoothed_pot_value = (alpha * raw_pot_value) +
((1 - alpha) * smoothed_pot_value)

            pot_value = int(smoothed_pot_value)

            # Map to angle and send to Arduino
            remapped_angle = int(remap(pot_value, 0, 1023, 0,
180))
```

```
                ser.write(f"{remapped_angle}\n".encode('utf-8'))

                # Update Plot Arrays
                y_vals.append(pot_value)
                y_vals.pop(0)
                x_vals.append(counter)
                x_vals.pop(0)
                counter += 1

                # Fast Plot Update
                line.set_ydata(y_vals)
                line.set_xdata(x_vals)
                ax.set_xlim(x_vals[0], x_vals[-1])

                fig.canvas.draw_idle()
                fig.canvas.flush_events()
    except KeyboardInterrupt:
        pass
    finally:
        ser.close()
        plt.ioff()
        plt.show()
        print("Serial connection closed.")
```

2.4 DATA COLLECTION & ANALYSIS

Potentiometer rotation	Servo angle	LED
At initial rotation	0	off
Half of the rotation	90	on
Full rotation	180	on

When the potentiometer turns until its initial point the servo will be at 0 degree and when we turn the potentiometer until its halfpoint the servo will be at 90 degree and the LED will light up upon reaching the halfpoint of the potentiometer. Finally, when the potentiometer reaches its maximum rotation the servo will be at 180 degree and the LED will stay on.

2.5 RESULTS & DISCUSSION

The servo manages to move according to the potentiometer rotation. When the potentiometer is at its initial point the reading should be zero and the servo should be at the 0 degree angle. The full rotation of the potentiometer will be the full 180 degree of the servo rotation. Based on that information we managed to plot a graph using the matplotlib in the python to

visualise the angle of the servo as the potentiometer rotated. We also managed to make an LED light up when the servo reached its half rotation.

One of the observations when doing the experiment is that we need to load the Arduino code first in the Arduino UNO board then we can upload the Python code into the board for the experiment to work. We were stuck forcing the board to work with just adjusting the code in Python not knowing that it would be related to the arduino code. After finding out about this information we managed to make everything according to the instructions given in the module.

2.6 CONCLUSION

The experiment successfully achieved its objective of establishing serial communication between the Arduino hardware and Python software. Moving from simple data acquisition to active bidirectional control highlights the versatility and effectiveness of this integration for mechatronic applications.

RECOMMENDATIONS

For future improvements, implementing robust error-handling in the Python script is recommended to safely manage unexpected serial disconnections or communication timeouts. Additionally, integrating data visualization libraries such as *matplotlib* would enhance the system by providing real-time graphical representations of the incoming sensor data.

REFERENCES

Prabhu, S. (2026, January 20). How Servo Motor Works & Interface It With Arduino. Last Minute Engineers. https://lastminuteengineers.com/servo-motor-arduino-tutorial/

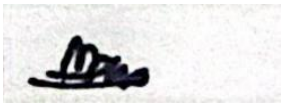
ACKNOWLEDGEMENT

Alhamdulillah, all praise and gratitude be to Allah SWT for His blessings, guidance, and mercy that enabled us to successfully complete this project entitled “Serial Communication between Arduino and Python”. Throughout the development process, we faced several challenges, but with His will and guidance, we managed to overcome them and complete the project successfully. We would also like to express our heartfelt appreciation to our lecturer for their continuous guidance, support, and valuable feedback throughout this project. Their advice has greatly helped us in understanding both the theoretical and practical aspects of microcontroller systems and serial communication.

STUDENT DECLARATION

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgments, and that the original work contained herein has not been undertaken or done by unauthorized sources or persons.

SIGNATURE	STUDENT DETAILS	CHECKLIST
	Name: DANISH IMAN BIN AJO Matric No: 2414273	Read [/] Understand [/] Agree [/]
	Name: ALIFF EMIR BIN AHMAD EZAD Matric No: 2414585	Read [/] Understand [/] Agree [/]
	Name:AHMAD AMIRULAQIL BIN ZAINORKHAIR Matric No: 2416879	Read [/] Understand [/] Agree [/]
	Name: MUHAMMAD IRFAN BIN ROSHDAN Matric No: 2418895	Read [/] Understand [/] Agree [/]